Harvesting the 174 image URLs

Two copy-pastes. Prepared 1 August 2026.

Why this cannot be done in the Rounds Codex session

The two Higgsfield tools that map a generation job to its image URL (`show_generations` and `job_display`) both render a client-side gallery widget. The cloud session has nowhere to draw one, so it refuses them outright. This is **not** a permission prompt waiting on you — the refusal is instant, no Higgsfield tool has ever been allow-listed, and every other Higgsfield tool runs there without one. Clicking "always allow" cannot fix it.

Route tried	Result
`show_generations`	Refused — twice, including after the connector reconnected
`job_display`	Refused
`sandbox_exec`	Refused
`show_medias`	Works, but returns 15 items — that is the **upload** library, not generations
The session transcript	`generate_image` returned job ids and "pending", no URLs
Deriving URLs from job ids	Unverifiable: no job map was saved for the 29 July batch, and the filename carries an `HHMMSS` that is not derivable

Step 1 — open a session that is not the cloud one

Go to **claude.ai** in a browser and start a new chat. Claude Code desktop or the CLI works equally well. What matters is only that it is not the Rounds Codex cloud session.

> **Check the Higgsfield connector is enabled for that chat.** It is a per-chat switch, not just per-account — a chat with it off will tell you it has no such tool, which looks identical to the problem above. Connector / tools menu in the new chat, Higgsfield on.

Step 2 — paste this in, exactly

```
Using the Higgsfield connector, call show_generations with size: 100
and type: "image". If the response has a non-null next_cursor, call it
again with that cursor, and keep going until next_cursor is null.

Then give me ONE JSON object and nothing else - no commentary, no
summary. Keys are each generation's id, values are that generation's
image URL (the first URL in its results). Skip any generation that has
no URL. It should look exactly like this:

{
"b5b6d40b-597b-4cef-aef6-cf00daf11857": "https://d8j0ntlcm91z4.clou
dfront.net/user_.../hf_20260731_....png",
"a073ad46-80c5-4b9d-8a07-2d71e7559411": "https://d8j0ntlcm91z4.clou
dfront.net/user_.../hf_20260731_....png"
}

Do not summarise, do not truncate, and do not reorder. I need every
pair.
```

Asking for id-and-URL only is deliberate. The full pages carry every generation prompt, which runs to megabytes and will not survive a paste; this form is about 20 KB for all 174.

Step 3 — paste the JSON back to me

Straight into the Rounds Codex chat, or saved as a file and attached. Either works. Then I run:

```
python3 tools/harvest_generations.py <file> --dry-run # look first
python3 tools/harvest_generations.py <file>
python3 tools/image_batch_plan.py --status
```

The harvester accepts both this compact form and raw `show_generations` pages, so if the other session hands you the full pages instead, that works too — just pass them along.

What to expect

- **166 of 174 pairs.** Eight failed at generation. The credit ledger showed 174 spends against 8 refunds landing within seconds of their own spend, which is the signature of a moderation refusal rather than a capacity failure. The harvester reports those as absent instead of quietly passing over them.
- The join is on **job id, never on order**. Pages come back newest-first and interleaved with other generations, and a positional match would attach a medical illustration to the wrong question — worse than a missing one.
- Re-running is safe. A second run records 0 new, and an existing different URL is refused unless `--force` is passed.
- **No deadline.** The URLs are unsigned CloudFront paths with no signature and no expiry parameter; the 29 July batch was still addressable two days later.

After the harvest

- `python3 tools/build_review_page.py` builds one standalone HTML page for review. Open it locally — it cannot be published as an Artifact, because that content policy blocks external hosts and these are remote images.
- The container still cannot fetch the images at all (the proxy blocks `higgsfield.ai` and the CDN), so the physician review happens in your browser, not in the session.
- Eight corrected re-fire prompts are staged in `tools/refire-queue.json` (16 credits). These are separate from whatever the harvest reports as failed, and need your go-ahead to spend.

Appendix A — the one other thing only you can do

Turn on Netlify's failed-deploy email. The Netlify API exposes no notifications endpoint, so this cannot be done from the session — I checked rather than assumed.

```
app.netlify.com -> rounds-codex -> Project configuration -> Notifications -> Add
notification -> Deploy failed -> Email
```

Deploys broke silently for 16 hours on 30/31 July and nothing alerted. This email is out-of-band from both the connector and any session, which is exactly why it is the fix that matters. The manual backstop is to load `rounds-codex.netlify.app/version.txt` after a deploy and check the timestamp moved.

Appendix B — when a connector looks broken

These three states present identically as "the connector is not working", and only one of them is worth re-authorising for. Ask me to run `ListConnectors` — it is one call and tells you which you are looking at.

Reads	Means	Do
`connected: true enabledInChat: false`	Authenticated, but switched off for that chat	Toggle it on in that chat's connector settings. **Do not re-authorise.**
`connected: false`	Genuinely deauthorised	claude.ai -> Settings -> Connectors -> Reconnect
`connected: null`	Status check unavailable	Unknown, not broken. Re-check; do not act on it.

On 1 August all four connectors — Google Drive, Higgsfield, Netlify and Supabase, 129 tools — dropped and returned as a single event, and Drive came back under a different server id. That is the shared transport reconnecting, not your account. It self-heals in a few minutes and there is no user-side fix; re-authorising during that window achieves nothing.